

caso, seja a posição de coluna 0 e linha 0 com 1 ciclo de clock e o pior caso seja a posição de coluna 3 e linha 3 com 16 ciclos de clock. A NAND ativa a 1 quando alguma posição é premida, ativando o Kpress.

A versão III percorre continuamente as linhas através do PENC, e vai varrendo da coluna 0 até à coluna 3. Nas linhas onde o botão não foi premido, o REG guarda 00. Quando o botão é premido, o GS ativa a 1 ativando também a 1 o Kpress, o REG guarda a linha onde o botão foi premida e forma-se o K3..0 com os valores da coluna vindos do Q1 e Q0 do Counter e com os valores de Y1 e Y0 do REG. O KScan está sempre a 1 pois o counter nunca para e o Reg guarda sempre um valor após uma coluna ter sido varrida (00 se linha onde botão não foi premido ou linha respetiva onde o botão foi premido). Fazendo com que o melhor caso, seja em qualquer posição da Coluna 0 com 1 ciclo clock e o Pior caso em qualquer posição da Coluna 3 com 4 ciclos clock.

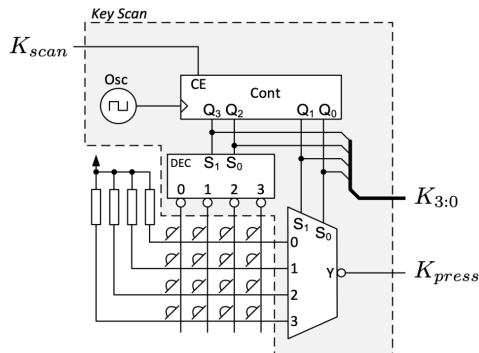


Figura 3: Diagrama de blocos do módulo *Key Scan*

O módulo KeyDelay é responsável pelo mecanismo de repetição de tecla que se após um determinado período, definido por *Tdelay* e conforme a Tabela 2c, a tecla se mantiver premida gera novo código da tecla.

A base da temporização assenta num contador de 4 bits, incrementado a cada pulso ativo do sinal de habilitação CE. Assumindo que CE é gerado com um período fixo (100 ms), o valor da contagem Q corresponde diretamente a múltiplos desse período.

Para identificar os instantes correspondentes a 500 ms, 1000 ms e 1500 ms, foram definidas três expressões lógicas combinacionais (TD_00, TD_01, TD_10), cada uma ativa apenas quando o contador

atinge o valor binário correspondente ao respetivo tempo.

Uma vez que um contador de 4 bits tem um alcance máximo de 15 contagens, insuficiente para representar o intervalo de 2000 ms, o circuito recorre a um segundo contador para estender a gama temporal.

Quando o primeiro contador atinge o seu valor máximo, o sinal de transporte CBO (Carry-Bit-Out) ativa um flip-flop, correspondendo ao 16º ciclo de clock, como se fosse uma alavanca, cuja saída passa a habilitar o segundo contador.

Este novo contador prossegue a contagem de mais 4 ciclos de clock para adicionar aos 16 já gerados pelo contador anterior, e a expressão TD_11 deteta o valor correspondente ao tempo adicional necessário para perfazer os 2000 ms totais.

Por fim, um MUX4X1 seleciona, com base no valor de Tdelay, qual dos quatro sinais decodificados (TD_00, TD_01, TD_10, TD_11) é encaminhado para a saída F.

O bloco *Key Control* foi implementado pela máquina de estados representada em *ASM-chart* na Figura 4.

Descrição da Máquina de estados do Key Control:

No estado *STANDING_BY*, o Kscan é ativado e o KeyDelay é mantido em reset. Se o Kpress estiver a 1, vai para o estado *READING_DATA* onde o Kval é ativado, enquanto que se o Kpress estiver a 0, vai manter-se no estado *STANDING_BY*.

No estado *READING_DATA*, o Kval é ativado. Se o Kack estiver a 1, vai para o estado *DATA_ACCEPTED*, onde o dado é aceite e o temporizador é habilitado, enquanto que se o Kack estiver a 0, vai manter-se no estado *READING_DATA*.

No estado *DATA_ACCEPTED*, o CEDelay é ativado, permitindo que o KeyDelay conte o tempo configurado pelo Tdelay. O estado mantém-se enquanto o Kpress estiver a 1 e o delay_out estiver a 0, ou seja, enquanto a tecla estiver pressionada e o tempo não tiver expirado. Se o Kpress passar a 0 ou o delay_out passar a 1, o estado seguinte será *STANDING_BY*, onde o KeyDelay é novamente colocado em reset.

A descrição hardware do bloco *Key Decode* em *VHDL* encontra-se no Anexo A.1.

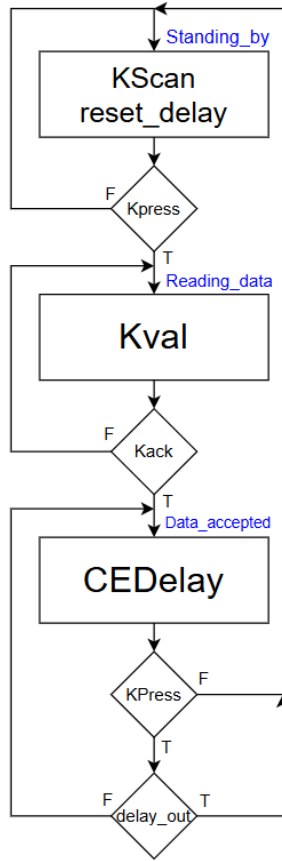


Figura 4: Máquina de estados do Key Control A.3

2 Ring Buffer

O bloco Ring Buffer é uma estrutura de dados de armazenamento temporário que funciona com uma disciplina FIFO (First In, First Out). No contexto do módulo Keyboard Reader, o principal objetivo é servir de memória intermédia (buffer) entre o produtor de dados (o bloco Key Decode, que deteta as teclas premidas) e o consumidor (o bloco Key Transmitter / módulo Control).

Esta estrutura garante o desacoplamento temporal entre o varrimento do teclado e a transmissão dos dados, permitindo reter até 16 palavras de 4 bits (códigos das teclas) caso o sistema consumidor não consiga processar as teclas imediatamente à velocidade com que são premidas.

O RingBuffer consiste em três blocos principais:

RAM : Núcleo de armazenamento do buffer. Implementado recorrendo a um descodificador de endereços (DecoderBuffer), 16 registos de 4 bits (Reg) para guardar as palavras e multiplexadores (MUX4) para encaminhar o dado lido para a saída.

Memory Address Control : Gestão dos ponteiros de leitura e escrita na memória RAM. Utiliza somadores de 4 bits para incrementar estes ponteiros (incPut e incGet) e aloja a lógica combinatória (RAMEF) que avalia se o buffer se encontra totalmente cheio ou completamente vazio. Contém 2 registos de 4 bits:

putIndex : Aponta para a próxima posição de memória onde será feito um registo (escrita);

getIndex : Aponta para a posição do dado mais antigo que está pronto para ser lido.

Ring Buffer Control : Unidade de controlo responsável por coordenar os protocolos de comunicação com o exterior e gerar os sinais de controlo internos (como a ativação da escrita na RAM wr ou a seleção do endereço através de putNget).

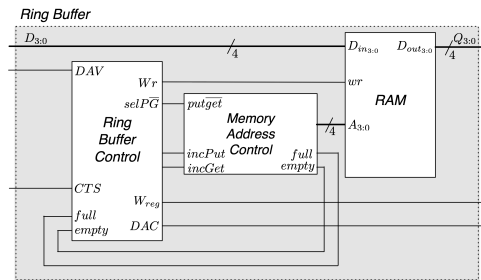


Figura 5: Diagrama de blocos do Ring Buffer

O bloco *Ring Buffer* foi implementado pela máquina de estados representada em *ASM-chart* na Figura 6.

Descrição da Máquina de estados do Ring Buffer:

No estado *STATE_IDLE*, a máquina aguarda por uma de duas condições. Se o *DAV* estiver a 1 e o *full* estiver a 0 (há dado disponível e o buffer não está cheio), vai para o estado *STATE_START_DACCEPT* para iniciar a receção. Se o *CTS* estiver a 1 e o *empty* estiver a

0 (o transmissor está pronto e há dado para enviar), vai para o estado `STATE_T_SEND_KEY` para iniciar o envio. Caso contrário, mantém-se no estado `STATE_IDLE`.

No estado `STATE_START_DACCEPT`, o `selPnG` é ativado, selecionando o ponteiro de escrita. A transição para `STATE_DACCEPT_ACK` é imediata, sem condições.

No estado `STATE_DACCEPT_ACK`, o `Wr` e o `selPnG` são ativados, escrevendo o dado no buffer. A transição para `STATE_AFTER_WRITE` é igualmente imediata.

No estado `STATE_AFTER_WRITE`, o `DAC` e o `incP` são ativados, sinalizando a aceitação do dado e incrementando o ponteiro de escrita. A transição para `STATE_WAIT_DAV_LOW` é imediata.

No estado `STATE_WAIT_DAV_LOW`, o `DAC` mantém-se ativo. A máquina aguarda que o `DAV` passe a 0, confirmando que o emissor reconheceu a aceitação. Enquanto o `DAV` estiver a 1, mantém-se neste estado. Quando o `DAV` passar a 0, regressa ao estado `STATE_IDLE`.

No estado `STATE_T_SEND_KEY`, o `Wreg` é ativado, carregando o dado a enviar. A transição para `STATE_T_INCG` é imediata.

No estado `STATE_T_INCG`, o `incG` é ativado, incrementando o ponteiro de leitura. A transição para `STATE_IDLE` é imediata.

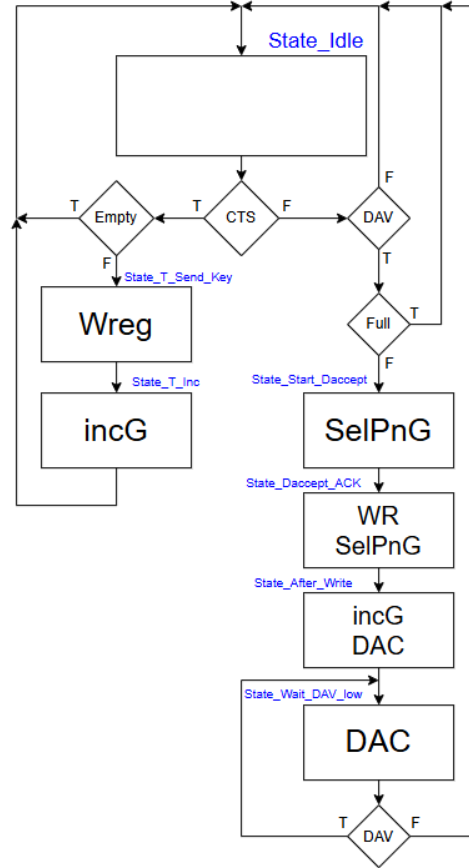


Figura 6: Máquina de estados do *Ring Buffer* A.13

3 Key Transmitter

Módulo responsável por realizar a conversão paralelo-série dos dados no subsistema Keyboard Reader. Tem como objetivo principal ler o código de 4 bits da tecla premida (disponibilizado em paralelo na sua entrada) e transmiti-lo de forma sequencial (série) através de uma única linha física de comunicação (TXD).

Esta transmissão é sincronizada por um sinal de relógio próprio (TXclk) e obedece a um protocolo estruturado em tramas de tamanho fixo, garantindo que o módulo recetor consiga reconstruir e interpretar os dados de forma fiável.

Cada transmissão de uma tecla gera uma trama com a duração exata de 8 períodos do relógio de transmissão (TXclk). O barramento série é gerido

através de um multiplexador de 8 canais que seleciona o bit a enviar em cada intervalo de tempo (slots de 0 a 7), mapeando a trama da seguinte forma:

Slot 0: TXD-INIT : Bit de inicialização/sincronismo. Assume o valor '0' enquanto a trama está a ser transmitida, servindo para sinalizar ao recetor o início do processo de comunicação;

Slot 1: START bit : Bit que inicia a '1';

Slots 2 a 5: Bits de Dados : Transmissão dos 4 bits do código da tecla (D(0) a D(3)), enviando primeiro o bit menos significativo (LSB) e por último o mais significativo (MSB);

Slot 6: STOP bit : Bit de paragem com o valor lógico '0';

Slot 7: Guard bit : Bit de salvaguarda com o valor lógico '1'. Quando o contador atinge este slot e o sinal TXclk está a '0', é gerada a flag de término (CFlag), sinalizando o fim da transmissão.

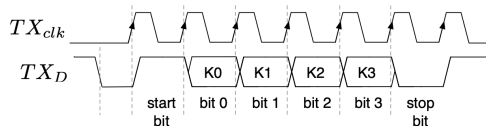


Figura 7: Protocolo de comunicação com o módulo *Key Transmitter*

O bloco *Key Transmitter* foi implementado pela máquina de estados representada em *ASM-chart* na Figura 8.

Descrição da Máquina de estados do Key Transmitter:

No estado *STATE_IDLE*, o contador de bits é mantido em reset (*RESET_C* ativo) e a linha *TXD_INIT* está a 1, indicando que a linha está em repouso. O *DAC* está a 1, sinalizando ao Key Decode que está livre para receber. Se o *DAV* passar a 1, indicando que há um dado disponível, vai para o estado *STATE_ACK*. Caso contrário, mantém-se no estado *STATE_IDLE*.

No estado *STATE_ACK*, o *ENR* é ativado, fazendo o latch do dado presente em *DataIn* para

o registo interno. O *DAC* passa a 0, sinalizando ao Key Decode que o dado foi aceite. A máquina aguarda que o *DAV* passe a 0, confirmando que o Key Decode reconheceu o acknowledgement. Enquanto o *DAV* se mantiver a 1, permanece no estado *STATE_ACK*. Quando o *DAV* passar a 0, avança para o estado *STATE_SEND*.

No estado *STATE_SEND*, o *TXD_INIT* passa a 0, indicando que a transmissão série está em curso, o *ENC* é ativado para habilitar o contador de bits, e o *DAC* mantém-se a 0. A máquina permanece neste estado até o *CFlag* passar a 1, sinalizando que o contador atingiu a contagem terminal e a frame foi completamente transmitida. Quando isso acontece, regressa ao estado *STATE_IDLE*, onde o contador é novamente colocado em reset e a linha *TXD_INIT* volta a 1.

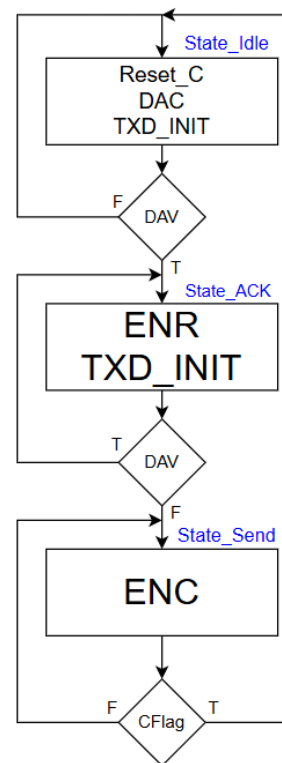


Figura 8: Máquina de estados do *Key Transmitter* A.7

4 Conclusões

O desenvolvimento do módulo Keyboard Reader concretizou com sucesso a captura e transmissão eficiente de dados na Ticket Machine. A arquitetura estrutural adotada provou ser robusta ao mitigar problemas críticos de temporização e interface física.

O bloco Key Decode eliminou eficazmente o bouncing do teclado por varrimento. O Ring Buffer (FIFO de 16 posições) introduziu um desacoplamento temporal crucial, salvaguardando os dados caso o sistema central apresente latência. Por fim, o Key Transmitter realizou a conversão paralelo-série estruturada em tramas de 8 slots, gerida por protocolos rígidos de handshake (DAV/DAC).

A VHDL

A.1 Key Decode

```
library ieee;
use ieee.std_logic_1164.all;

entity Key_Decode is
  port (
    Kack, RESET, CLK          : in  std_logic;
    Tdelay                    : in  std_logic_vector(1 downto 0);
    Kval, clk_out, clk_out_rise : out std_logic;
    K                          : out std_logic_vector(3 downto 0);
    Keys_Vertical              : out std_logic_vector(3 downto 0);
    Keys_Horizontal            : in  std_logic_vector(3 downto 0)
  );
end entity Key_Decode;

architecture logicFunction of Key_Decode is

  component Key_Control is
    port (
      CLK, RESET, Kack, Kpress      : in  std_logic;
      Tdelay                        : in  std_logic_vector(1 downto 0);
      Kval, Kscan, clk_out, clk_out_rise : out std_logic
    );
  end component Key_Control;

  component Key_Scan is
    port (
      Kscan, CLK, RESET : in  std_logic;
      Keys_Vertical      : out std_logic_vector(3 downto 0);
      Keys_Horizontal     : in  std_logic_vector(3 downto 0);
      K                    : out std_logic_vector(3 downto 0);
      Kpress               : out std_logic
    );
  end component Key_Scan;

  component CLKDIV is
    generic (
      div : natural := 50000000
    );
    port (
      clk_in  : in  std_logic;
      clk_out : out std_logic
    );
  end component CLKDIV;

  signal Kpress, Kscan, CLK_Divider : std_logic;

begin

  Clk_div: component CLKDIV
  generic map (
    div => 50000000
  )
  port map (
    clk_in  => CLK,
    clk_out => CLK_Divider -- 100ms, 10Hz
  );

  scan: component Key_Scan
  port map (
    RESET      => RESET,
```

```
        CLK          => CLK_Divider,
        Kscan         => Kscan,
        Kpress        => Kpress,
        K             => K,
        Keys_Vertical => Keys_Vertical,
        Keys_Horizontal => Keys_Horizontal
    );

    control: component Key_Control
    port map (
        RESET          => RESET,
        CLK            => not CLK_Divider,
        Kpress         => Kpress,
        Kack           => Kack,
        Tdelay         => Tdelay,
        Kval           => Kval,
        Kscan          => Kscan,
        clk_out        => clk_out,
        clk_out_rise   => clk_out_rise
    );

end architecture logicFunction;
```

A.2 Key Scan

```
library IEEE;
use IEEE.std_logic_1164.all;

entity Key_Scan is

    port (
        Kscan, CLK, RESET : in  std_logic;
        Keys_Vertical      : out std_logic_vector(3 downto 0);
        Keys_Horizontal    : in  std_logic_vector(3 downto 0);
        K                  : out std_logic_vector(3 downto 0);
        Kpress             : out std_logic
    );

end entity Key_Scan;

architecture scan of Key_Scan is

    component Counter is
        port (
            CE    : in  std_logic;
            CLK   : in  std_logic;
            Q      : out std_logic_vector(3 downto 0);
            RESET : in  std_logic
        );
    end component Counter;

    component Decoder is
        port (
            S : in  std_logic_vector(1 downto 0);
            C : out std_logic_vector(3 downto 0)
        );
    end component Decoder;

    component MUX4X1 is
        port (
            A : in  std_logic_vector(3 downto 0);
            OP : in  std_logic_vector(1 downto 0);
            F : out std_logic
        );
    end component MUX4X1;

    signal counter_out, not_c : std_logic_vector(3 downto 0);

    signal not_k_press : std_logic;

begin

    Kcounter: component Counter
    port map (
        CLK   => CLK,
        RESET => RESET,
        CE    => Kscan,
        Q      => counter_out
    );

    mux: component MUX4X1
    port map (
        A      => Keys_Horizontal,
        OP     => counter_out(1 downto 0),
        F      => not_k_press
    );

    SKdecoder: component Decoder
```

```
port map (  
    S    => counter_out(3 downto 2),  
    C    => not_c  
);  
  
Kpress      <= not not_k_press;  
Keys_Vertical <= not not_c;  
K           <= counter_out;  
  
end architecture scan;
```

A.3 Key Decoder FSM

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity KeyDecoderFSM is
    port (
        reset          : in  std_logic;
        clk            : in  std_logic;
        Kpress, Kack    : in  std_logic;
        Tdelay         : in  std_logic_vector(1 downto 0); -- 00=500ms 01=1000ms
        -- 10=1500ms 11=2000ms
        Kval, Kscan, clk_out, clk_out_rise : out std_logic
    );
end entity KeyDecoderFSM;

architecture behavioral of KeyDecoderFSM is

    component KeyDelay is
        port (
            CLK, CE, RESET : in  std_logic;
            Tdelay         : in  std_logic_vector(1 downto 0);
            F              : out std_logic
        );
    end component KeyDelay;

    component CLKDIV is
        generic (
            div : natural := 50000000
        );
        port (
            clk_in  : in  std_logic;
            clk_out : out std_logic
        );
    end component CLKDIV;

    component EdgeDetect is
        port (
            CLK : in  std_logic;
            RESET : in  std_logic;
            SIG_IN : in  std_logic;
            RISE : out std_logic
        );
    end component EdgeDetect;

    type STATE_TYPE is (STANDING_BY, READING_DATA, DATA_ACCEPTED);

    signal CurrentState, NextState : STATE_TYPE;
    signal clk_out1                : std_logic;
    signal clk_out_prev            : std_logic := '0';
    signal clk_out_rise1           : std_logic;

    signal resetdelay              : std_logic;
    signal CEDelay                 : std_logic;
    signal delay_out               : std_logic;

begin

    clk_out <= delay_out;

    -- Instantiate delay clock generator
    clkd: component KeyDelay

```

```
port map (
    CLK    => CLK,
    CE     => CEDelay,
    Tdelay => Tdelay,
    F      => delay_out,
    RESET  => resetdelay
);

StateRegister: process (clk, reset) is
begin
    if reset = '1' then
        CurrentState <= STANDING_BY;
    elsif rising_edge(clk) then
        CurrentState <= NextState;
    end if;
end process StateRegister;

GenerateNextState: process (CurrentState, Kpress, Kack, delay_out) is
begin
    case CurrentState is
        when STANDING_BY =>
            if (Kpress = '1') then
                NextState <= READING_DATA;
            else
                NextState <= STANDING_BY;
            end if;

        when READING_DATA =>
            if (Kack = '1') then
                NextState <= DATA_ACCEPTED;
            else
                NextState <= READING_DATA;
            end if;

        when DATA_ACCEPTED =>
            if Kpress = '0' or delay_out = '1' then
                NextState <= STANDING_BY;
            else
                NextState <= DATA_ACCEPTED;
            end if;

    end case;
end process GenerateNextState;

Kscan <= '1' when (CurrentState = STANDING_BY) else '0';
Kval  <= '1' when (CurrentState = READING_DATA) else '0';
resetdelay <= '1' when (CurrentState = STANDING_BY) else '0';
CEDelay <= '1' when (CurrentState = DATA_ACCEPTED) else '0';
end architecture behavioral;
```

A.4 Key Control

```
library ieee;
use ieee.std_logic_1164.all;

entity Key_Control is
    port (
        Kack, Kpress, RESET, CLK          : in  std_logic;
        Tdelay                             : in  std_logic_vector(1 downto 0);
        Kval, Kscan, clk_out, clk_out_rise : out std_logic
    );
end entity Key_Control;

architecture logicFunction of Key_Control is

    component KeyDecoderFSM is
        port (
            reset          : in  std_logic;
            clk             : in  std_logic;
            Kpress, Kack    : in  std_logic;
            Tdelay          : in  std_logic_vector(1 downto 0);
            Kval, Kscan, clk_out, clk_out_rise : out std_logic
        );
    end component KeyDecoderFSM;

begin

    fsm: component KeyDecoderFSM
    port map (
        reset      => RESET,
        clk        => CLK,
        Kpress     => Kpress,
        Kack       => Kack,
        Tdelay     => Tdelay,
        Kval       => Kval,
        Kscan      => Kscan,
        clk_out    => clk_out,
        clk_out_rise => clk_out_rise
    );
end architecture logicFunction;
```

A.5 Key Delay

```

library ieee;
use ieee.std_logic_1164.all;

entity KeyDelay is
  port (
    CLK, CE, RESET : in  std_logic;
    Tdelay          : in  std_logic_vector(1 downto 0);
    F               : out std_logic
  );
end entity KeyDelay;

architecture logicFunction of KeyDelay is

  component Counter is
    port (
      CE      : in  std_logic;
      CLK     : in  std_logic;
      Q       : out std_logic_vector(3 downto 0);
      RESET   : in  std_logic;
      CBO     : out std_logic
    );
  end component Counter;

  component MUX4X1 is
    port (
      A : in  std_logic_vector(3 downto 0);
      OP : in std_logic_vector(1 downto 0);
      F : out std_logic
    );
  end component MUX4X1;

  component FFD is
    port (
      CLK : in  std_logic;
      RESET : in STD_LOGIC;
      SET  : in  std_logic;
      D    : in  STD_LOGIC;
      EN   : in  STD_LOGIC;
      Q    : out std_logic
    );
  end component FFD;

  signal Q, Y      : std_logic_vector(3 downto 0);
  signal CBO, C    : std_logic;

  signal TD_00,TD_01,TD_10,TD_11 :std_logic;

begin

  Count: component Counter
  port map (
    CE      => CE,
    CLK     => CLK,
    Q       => Q,
    RESET   => RESET,
    CBO     => CBO
  );

  Count11: component Counter
  port map (
    CE      => C,

```

```
        CLK    => CLK,
        Q      => Y,
        RESET  => RESET
    );

    alavanca: component FFD
port map (
    CLK    => CLK,
    RESET  => RESET,
    set    => '0',
    D      => '1',
    EN     => CBO,
    Q      => C
);

mux: component MUX4X1
port map (
    A(0)    => TD_00,
    A(1)    => TD_01,
    A(2)    => TD_10,
    A(3)    => TD_11,
    OP      => Tdelay,
    F       => F
);

    TD_00 <= not Q(3) and Q(2) and not Q(1) and Q(0);
    TD_01 <= Q(3) and not Q(2) and Q(1) and not Q(0);
    TD_10 <= Q(3) and Q(2) and Q(1) and Q(0);
    TD_11 <= not Y(3) and Y(2) and not Y(1) and not Y(0);

end architecture logicFunction;
```

A.6 Keyboard Reader

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity KeyboardReader is
    port (
        CLK                : in  STD_LOGIC;
        RESET              : in  STD_LOGIC;

        Tdelay              : in  STD_LOGIC_VECTOR(1 downto 0);

        Keys_Vertical       : out STD_LOGIC_VECTOR(3 downto 0);
        Keys_Horizontal     : in  STD_LOGIC_VECTOR(3 downto 0);

        TXclk               : in  STD_LOGIC;
        TXD, clk_out, clk_out_rise : out STD_LOGIC
    );
end entity KeyboardReader;

architecture logicFunction of KeyboardReader is
    component Key_Decode is
        port (
            Kack, RESET, CLK          : in  STD_LOGIC;
            Tdelay                    : in  STD_LOGIC_VECTOR(1 downto 0);
            Kval, clk_out, clk_out_rise : out STD_LOGIC;
            K                          : out STD_LOGIC_VECTOR(3 downto 0);
            Keys_Vertical              : out STD_LOGIC_VECTOR(3 downto 0);
            Keys_Horizontal            : in  STD_LOGIC_VECTOR(3 downto 0)
        );
    end component Key_Decode;

    component RingBuffer is
        port (
            CLK : in  std_logic;
            RESET : in  std_logic;
            CTS : in  std_logic;
            DAV : in  std_logic;
            D : in  std_logic_vector(3 downto 0);
            Q : out std_logic_vector(3 downto 0);
            Wreg : out std_logic;
            DAC : out std_logic
        );
    end component RingBuffer;

    component KeyTransmitter is
        port (
            CLK : in  STD_LOGIC;
            RESET : in  STD_LOGIC;
            DataIn : in  STD_LOGIC_VECTOR(3 downto 0);
            Load : in  STD_LOGIC;
            KBfree : out STD_LOGIC;
            TXclk : in  STD_LOGIC;
            TXD : out STD_LOGIC
        );
    end component KeyTransmitter;

    signal KEY_CODE_LINK, KEY_VALUE : STD_LOGIC_VECTOR(3 downto 0);
    signal KVAL_LINK                 : STD_LOGIC; -- Kval level from Key_Decode
    signal KACK_LINK                 : STD_LOGIC; -- Kack pulse back to Key_Decode
    signal LOAD_LINK                 : STD_LOGIC; -- single-cycle Load to KeyTransmitter
    signal KBFREE_LINK               : STD_LOGIC; -- transmitter idle / done

```

```
begin
```

```
DECODE: component Key_Decode
port map (
    CLK          => CLK,
    RESET        => RESET,
    Kack         => KACK_LINK,
    Tdelay       => Tdelay,
    Kval         => KVAL_LINK,
    K            => KEY_CODE_LINK,
    Keys_Vertical => Keys_Vertical,
    Keys_Horizontal => Keys_Horizontal,
    clk_out      => clk_out,
    clk_out_rise  => clk_out_rise
);
```

```
buff: component RingBuffer
port map (
    CLK          => CLK,
    RESET        => RESET,
    CTS          => KBFREE_LINK,
    DAV          => KVAL_LINK,
    D            => KEY_CODE_LINK,
    Q            => KEY_VALUE,
    Wreg         => LOAD_LINK,
    DAC          => KACK_LINK
);
```

```
TRANSMITTER: component KeyTransmitter
port map (
    CLK          => CLK,
    RESET        => RESET,
    DataIn       => KEY_VALUE,
    Load        => LOAD_LINK,
    KBfree       => KBFREE_LINK,
    TXclk        => TXclk,
    TXD          => TXD
);
```

```
end architecture logicFunction;
```

A.7 Key Trasmitter Control

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

-- =====
-- KeyTransmitter_Control
--
-- FSM that controls the serial transmission of a 4-bit key code.
--
-- States:
-- STATE_IDLE : Waiting for a valid key (DAV = '1')
-- STATE_ACK  : Acknowledges the key to Key Decode, latches data
-- STATE_SEND : Clocks out the serial frame until CFlag = '1'
--
-- Outputs:
-- ENR       : Enable Register (latch DataIn into Reg)
-- RESET_C   : Reset the bit counter (active HIGH)
-- ENC       : Enable bit counter
-- DAC       : Data Acknowledged (fed back to Key Decode as KBfree)
-- TXD_INIT  : '1' = line idle/high, '0' = frame in progress
-- =====

entity KeyTransmitter_Control is
    port (
        CLK       : in  STD_LOGIC;
        RESET     : in  STD_LOGIC;
        DAV       : in  STD_LOGIC; -- Data Available (Load from Key Decode)
        CFlag     : in  STD_LOGIC; -- Terminal count flag (frame complete)

        ENR       : out STD_LOGIC; -- Enable data register
        RESET_C   : out STD_LOGIC; -- Reset bit counter
        ENC       : out STD_LOGIC; -- Enable bit counter
        DAC       : out STD_LOGIC; -- Data Acknowledged
        TXD_INIT  : out STD_LOGIC; -- TXD line value ('1'=idle, '0'=sending)
    );
end entity KeyTransmitter_Control;

architecture behavioral of KeyTransmitter_Control is
    type STATE_TYPE is (STATE_IDLE, STATE_ACK, STATE_SEND);
    signal CurrentState, NextState : STATE_TYPE;

begin
    -- =====
    -- State register
    -- =====
    StateRegister: process (CLK, RESET) is
    begin
        if RESET = '1' then
            CurrentState <= STATE_IDLE;
        elsif rising_edge(CLK) then
            CurrentState <= NextState;
        end if;
    end process StateRegister;

    -- =====
    -- Next-state logic
    -- =====
    GenerateNextState: process (CurrentState, DAV, CFlag) is
    begin

```

```
case CurrentState is

    -- Wait for Key Decode to signal a valid key
    when STATE_IDLE =>
        if DAV = '1' then
            NextState <= STATE_ACK;
        else
            NextState <= STATE_IDLE;
        end if;

    -- Latch data and acknowledge; wait for DAV to drop
    when STATE_ACK =>
        if DAV = '0' then
            NextState <= STATE_SEND;
        else
            NextState <= STATE_ACK;
        end if;

    -- Transmit serial frame; leave when counter reaches terminal count
    when STATE_SEND =>
        if CFlag = '1' then
            NextState <= STATE_IDLE;
        else
            NextState <= STATE_SEND;
        end if;

end case;
end process GenerateNextState;

-----

-- Output logic (Moore)
-----

-- Latch DataIn into the Reg while acknowledging the key
ENR <= '1' when CurrentState = STATE_ACK else '0';

-- Hold counter in reset outside of the SEND state
RESET_C <= '1' when CurrentState = STATE_IDLE else '0';

-- Inform Key Decode the key has been accepted
DAC <= '0' when CurrentState = STATE_ACK or CurrentState = STATE_SEND else '1';

-- Pull TXD low while the frame is being transmitted
TXD_INIT <= '0' when CurrentState = STATE_SEND else '1';

-- Run the bit counter only while sending
ENC <= '1' when CurrentState = STATE_SEND else '0';

end architecture behavioral;
```

A.8 Key Transmitter

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

-- =====
-- KeyTransmitter
--
-- Serialises a 4-bit key code over a single TXD line, clocked
-- by an external TXclk signal. Based on Francisco's
-- KEY_TRANSMITTER design, adapted to use Common/ components.
--
-- Serial frame (8 TXclk periods):
--
-- Counter  Mux slot  TXD value
-- -----
-- 0         M(0)     TXD_INIT ('0' while STATE_SEND, init/sync bit)
-- 1         M(1)     '1'      (START bit)
-- 2         M(2)     D(0)      (LSB of key code)
-- 3         M(3)     D(1)
-- 4         M(4)     D(2)
-- 5         M(5)     D(3)      (MSB of key code)
-- 6         M(6)     '0'      (STOP bit)
-- 7         M(7)     '1'      (placeholder / frame guard)
--
-- CFlag goes high at counter=7 while TXclk='0', triggering the
-- control FSM to return to STATE_IDLE.
-- =====

entity KeyTransmitter is

    port (
        CLK      : in  STD_LOGIC;
        RESET    : in  STD_LOGIC;

        DataIn   : in  STD_LOGIC_VECTOR(3 downto 0); -- 4-bit key code
        Load     : in  STD_LOGIC; -- pulse: key is valid
        KBfree   : out STD_LOGIC; -- '1' when ready for next key

        TXclk    : in  STD_LOGIC; -- serial clock (from host)
        TXD      : out STD_LOGIC  -- serial data output
    );

end entity KeyTransmitter;

architecture behavioral of KeyTransmitter is

    -- -----
    -- Component declarations
    -- -----

    component KeyTransmitter_Control is
        port (
            CLK      : in  STD_LOGIC;
            RESET    : in  STD_LOGIC;
            DAV      : in  STD_LOGIC;
            CFlag    : in  STD_LOGIC;
            ENR      : out STD_LOGIC;
            RESET_C  : out STD_LOGIC;
            ENC      : out STD_LOGIC;
            DAC      : out STD_LOGIC;
            TXD_INIT : out STD_LOGIC
        );
    end component KeyTransmitter_Control;

```

```

-- 4-bit register from Common/
component Reg is
  port (
    CLK   : in  STD_LOGIC;
    RESET : in  STD_LOGIC;
    D     : in  STD_LOGIC_VECTOR(3 downto 0);
    EN    : in  STD_LOGIC;
    Q     : out STD_LOGIC_VECTOR(3 downto 0)
  );
end component Reg;

-- 4-bit up counter from Common/ (CounterDown.vhd entity name is Counter)
component Counter is
  port (
    CE    : in  STD_LOGIC;
    CLK   : in  STD_LOGIC;
    Q     : out STD_LOGIC_VECTOR(3 downto 0);
    RESET : in  STD_LOGIC
  );
end component Counter;

-- 8-to-1 mux with 3-bit select
component Mux8x3 is
  port (
    A : in  STD_LOGIC_VECTOR(7 downto 0);
    S : in  STD_LOGIC_VECTOR(2 downto 0);
    O : out STD_LOGIC
  );
end component Mux8x3;

-----
-- Internal signals
-----

-- Control FSM outputs
signal ENR_LINK   : STD_LOGIC;
signal RESET_C_LINK : STD_LOGIC;
signal ENC_LINK   : STD_LOGIC;
signal TXD_INIT_LINK : STD_LOGIC;

-- Registered key code
signal Q_LINK : STD_LOGIC_VECTOR(3 downto 0);

-- Bit counter output
signal O_LINK : STD_LOGIC_VECTOR(3 downto 0);

-- 3-bit mux select (lower 3 bits of counter)
signal S_LINK : STD_LOGIC_VECTOR(2 downto 0);

-- 8-slot mux input vector
signal M_LINK : STD_LOGIC_VECTOR(7 downto 0);

-- Terminal count flag: counter=7 and TXclk is LOW
signal C_FLAG : STD_LOGIC;

begin

-----
-- Terminal count: frame complete when counter reaches 7
-- and TXclk is LOW (matches Francisco's original condition)
-----
C_FLAG <= (O_LINK(2) and O_LINK(1) and O_LINK(0)) and not TXclk;

```

```

-----
-- Control FSM
-----

M0: component KeyTransmitter_Control
port map (
    CLK      => CLK,
    RESET    => RESET,
    DAV      => Load,
    CFlag    => C_FLAG,
    ENR      => ENR_LINK,
    RESET_C  => RESET_C_LINK,
    ENC      => ENC_LINK,
    DAC      => KBfree,
    TXD_INIT => TXD_INIT_LINK
);

-----
-- Data register: latches DataIn when ENR is asserted
-- Uses Common/Reg.vhd (replaces Francisco's REGISTER4BITS)
-----

M1: component Reg
port map (
    CLK      => CLK,
    RESET    => RESET,
    D        => DataIn,
    EN       => ENR_LINK,
    Q        => Q_LINK
);

-----
-- Bit counter: counts TXclk rising edges during transmission
-- Uses Common/Counter (CounterDown.vhd, actually an up counter)
-- CLK driven by TXclk so it advances once per serial clock
-- RESET driven by RESET_C (active-high, matches Common/Counter)
-- CE driven by ENC (enabled only during STATE_SEND)
-----

M2: component Counter
port map (
    CLK      => TXclk,
    RESET    => RESET_C_LINK,
    CE       => ENC_LINK,
    Q        => O_LINK
);

-----
-- Mux select: lower 3 bits of counter choose the frame slot
-----

S_LINK(2 downto 0) <= O_LINK(2 downto 0);

-----
-- Serial frame definition (8 slots)
-----

M_LINK(0) <= TXD_INIT_LINK; -- Slot 0 : INIT bit ('0' while sending)
M_LINK(1) <= '1';           -- Slot 1 : START bit
M_LINK(2) <= Q_LINK(0);      -- Slot 2 : D(0) / LSB
M_LINK(3) <= Q_LINK(1);      -- Slot 3 : D(1)
M_LINK(4) <= Q_LINK(2);      -- Slot 4 : D(2)
M_LINK(5) <= Q_LINK(3);      -- Slot 5 : D(3) / MSB
M_LINK(6) <= '0';           -- Slot 6 : STOP bit
M_LINK(7) <= '1';           -- Slot 7 : placeholder / frame guard

-----
-- Output mux: selects the correct frame slot onto TXD
-----

```

```
M3: component Mux8x3
port map (
    A      => M_LINK,
    S      => S_LINK,
    O      => TXD
);

end architecture behavioral;
```

A.9 Ring Buffer

```
library ieee;
use ieee.std_logic_1164.all;

entity RingBuffer is
  port (
    CLK      : in  std_logic;
    RESET    : in  std_logic;
    CTS      : in  std_logic;
    DAV      : in  std_logic;
    D        : in  std_logic_vector(3 downto 0);
    Q        : out std_logic_vector(3 downto 0);
    Wreg     : out std_logic;
    DAC      : out std_logic
  );
end entity RingBuffer;

architecture logicFunction of RingBuffer is
  component MAC is
    port (
      CLK      : in  std_logic;
      RESET    : in  std_logic;
      putNget  : in  std_logic;
      incPut   : in  std_logic;
      incGet   : in  std_logic;
      Addr     : out std_logic_vector(3 downto 0);
      full     : out std_logic;
      empty    : out std_logic
    );
  end component MAC;

  component RAM is
    port (
      CLK      : in  std_logic;
      RESET    : in  std_logic;
      address  : in  std_logic_vector(3 downto 0);
      wr       : in  std_logic;
      din      : in  std_logic_vector(3 downto 0);
      dout     : out std_logic_vector(3 downto 0)
    );
  end component RAM;

  component RBC is
    port (
      CLK      : in  std_logic;
      RESET    : in  std_logic;
      DAV, CTS  : in  std_logic;
      full, empty : in std_logic;

      Wr       : out std_logic;
      selPnG   : out std_logic;
      Wreg     : out std_logic;
      DAC      : out std_logic;
      incP, incG : out std_logic
    );
  end component RBC;

  signal addrRam : std_logic_vector(3 downto 0);

  signal fullS, emptyS, incPutS, incGetS, putNgetS, wrS : std_logic;
begin
```

```
control: component RBC
port map (
    CLK      => CLK,
    RESET    => RESET,
    DAV      => DAV,
    CTS      => CTS,
    full     => fullS,
    empty    => emptyS,
    Wr       => wrS,
    selPnG   => putNgetS,
    Wreg     => Wreg,
    DAC      => DAC,
    incP     => incPutS,
    incG     => incGetS
);

memControl: component MAC
port map (
    CLK      => CLK,
    RESET    => RESET,
    putNget  => putNgetS,
    incPut   => incPutS,
    incGet   => incGetS,
    Addr     => addrRam,
    full     => fullS,
    empty    => emptyS
);

mem: component RAM
port map (
    CLK      => CLK,
    RESET    => RESET,
    address  => addrRam,
    wr       => wrS,
    din      => D,
    dout     => Q
);

end architecture logicFunction;
```

A.10 MAC

```
library ieee;
use ieee.std_logic_1164.all;

entity MAC is
  port (
    CLK      : in  std_logic;
    RESET    : in  std_logic;
    putNget  : in  std_logic;
    incPut   : in  std_logic;
    incGet   : in  std_logic;
    Addr     : out std_logic_vector(3 downto 0);
    full     : out std_logic;
    empty    : out std_logic
  );
end entity MAC;

architecture logicFunction of MAC is
  component Reg is
    port (
      CLK      : in  std_logic;
      RESET    : in  std_logic;
      D        : in  std_logic_vector(3 downto 0);
      EN       : in  std_logic;
      Q        : out std_logic_vector(3 downto 0)
    );
  end component Reg;

  component sumador_4bit is
    port (
      A, B : in  std_logic_vector(3 downto 0);
      R    : out std_logic_vector(3 downto 0);
      Ci   : in  std_logic;
      Co   : out std_logic
    );
  end component sumador_4bit;

  component MUX4 is
    port (
      A : in  std_logic_vector(3 downto 0);
      B : in  std_logic_vector(3 downto 0);
      OP : in  std_logic;
      F : out std_logic_vector(3 downto 0)
    );
  end component MUX4;

  component RAMEF is
    port (
      CLK      : in  std_logic;
      RESET    : in  std_logic;
      inc, dec  : in  std_logic;
      full, empty : out std_logic
    );
  end component RAMEF;

  signal putI, putIAdder, getI, getIAdder : std_logic_vector(3 downto 0);
begin

  incPutSum: component sumador_4bit
  port map (
    A => putI,
    B => "0000",
    R => putIAdder,
```

```
        Ci      => incPut
    );
    putIndex: component Reg
    port map (
        CLK      => CLK,
        RESET    => RESET,
        D        => putIAdder,
        EN       => '1',
        Q        => putI
    );

    incGetSum: component sumador_4bit
    port map (
        A        => getI,
        B        => "0000",
        R        => getIAdder,
        Ci       => incGet
    );
    getIndex: component Reg
    port map (
        CLK      => CLK,
        RESET    => RESET,
        D        => getIAdder,
        EN       => '1',
        Q        => getI
    );

    m: component MUX4
    port map (
        A        => getI,
        B        => putI,
        OP       => putNget,
        F        => Addr
    );

    ef: component RAMEF
    port map (
        CLK      => CLK,
        RESET    => RESET,
        inc      => incPut,
        dec      => incGet,
        full     => full,
        empty    => empty
    );

end architecture logicFunction;
```

A.11 Mux8x3

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

-- =====
-- Mux8x3
--
-- 8-to-1 multiplexer with a 3-bit select.
-- Selects one bit from input vector A based on S.
--
-- S = "000" -> O = A(0)
-- S = "001" -> O = A(1)
-- ...
-- S = "111" -> O = A(7)
-- =====

entity Mux8x3 is
    port (
        A : in  STD_LOGIC_VECTOR(7 downto 0);
        S : in  STD_LOGIC_VECTOR(2 downto 0);
        O : out STD_LOGIC
    );
end entity Mux8x3;

architecture logicFunction of Mux8x3 is
begin
    O <=
        (A(0) and not S(2) and not S(1) and not S(0)) or (A(1) and not S(2) and not S(1) and S(0)) or
        (A(2) and not S(2) and S(1) and not S(0)) or (A(3) and not S(2) and S(1) and S(0)) or
        (A(4) and S(2) and not S(1) and not S(0)) or (A(5) and S(2) and not S(1) and S(0)) or
        (A(6) and S(2) and S(1) and not S(0)) or (A(7) and S(2) and S(1) and S(0));
end architecture logicFunction;
```

A.12 RAMEF

```
library ieee;
use ieee.std_logic_1164.all;

entity RAMEF is
    port (
        CLK      : in  std_logic;
        RESET    : in  std_logic;
        inc, dec  : in  std_logic;
        full, empty : out std_logic
    );
end entity RAMEF;

architecture structural of RAMEF is

    component Reg is
        port (
            CLK : in  std_logic;
            RESET : in  std_logic;
            D : in  std_logic_vector(3 downto 0);
            EN : in  std_logic;
            Q : out std_logic_vector(3 downto 0)
        );
    end component Reg;

    component AdderSubtrator is
        port (
            A : in  std_logic_vector(3 downto 0);
            B : in  std_logic_vector(3 downto 0);
            CBi, OPau : in  std_logic;
            iCBO : out std_logic;
            R : out std_logic_vector(3 downto 0)
        );
    end component AdderSubtrator;

    component MUX4 is
        port (
            A, B : in  std_logic_vector(3 downto 0);
            OP : in  std_logic;
            F : out std_logic_vector(3 downto 0)
        );
    end component MUX4;

    component FFD is
        port (
            CLK : in  std_logic;
            RESET : in  std_logic;
            SET : in  std_logic;
            D : in  std_logic;
            EN : in  std_logic;
            Q : out std_logic
        );
    end component FFD;

    signal count_q : std_logic_vector(3 downto 0);
    signal addsub : std_logic_vector(3 downto 0);
    signal next_count : std_logic_vector(3 downto 0);
    signal en_reg : std_logic;
    signal CBO : std_logic;
    signal CBO_reg : std_logic;

begin
```

```

U_C: component AdderSubtrator
port map (
    A    => count_q,
    B    => "0000",
    CBi  => '1',
    OPau => dec,
    R    => addsub,
        iCBO => CBO
);

U_MUX_HOLD: component MUX4
port map (
    A    => count_q,
    B    => addsub,
    OP    => en_reg,
    F    => next_count
);

en_reg <= inc xor dec;

U_REG: component Reg
port map (
    CLK  => CLK,
    RESET => RESET,
    D    => next_count,
    EN   => en_reg,
    Q    => count_q
);

U_CBO_REG: component FFD
port map (
    CLK    => CLK,
    RESET  => RESET,
    SET    => '0',
    D      => CBO,
    EN     => en_reg,
    Q      => CBO_reg
);

full    <= CBO_reg;
empty   <= ((not count_q(3)) and (not count_q(2)) and (not count_q(1)) and (not count_q(0))) and not
    ↪ CBO_reg;
end architecture structural;

```

A.13 RBC

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity RBC is

    port (
        CLK          : in  std_logic;
        RESET        : in  std_logic;
        DAV, CTS      : in  std_logic;
        full, empty   : in  std_logic;

        Wr           : out std_logic;
        selPnG       : out std_logic;
        Wreg         : out std_logic;
        DAC          : out std_logic;
        incP, incG    : out std_logic
    );

end entity RBC;

architecture behavioral of RBC is

    type STATE_TYPE
    is (
        STATE_IDLE,
        STATE_START_DACCEPT,
        STATE_DACCEPT_ACK,
        STATE_AFTER_WRITE,
        STATE_WAIT_DAV_LOW,
        STATE_T_SEND_KEY,
        STATE_T_INCG
    );
    signal CurrentState, NextState : STATE_TYPE;

begin

    StateRegister: process (CLK, RESET) is
    begin
        if RESET = '1' then
            CurrentState <= STATE_IDLE;
        elsif rising_edge(CLK) then
            CurrentState <= NextState;
        end if;
    end process StateRegister;

    GenerateNextState: process (CurrentState, DAV, CTS, full, empty) is
    begin
        case CurrentState is
            when STATE_IDLE =>
                if DAV = '1' and full = '0' then
                    NextState <= STATE_START_DACCEPT;
                elsif CTS = '1' and empty = '0' then
                    NextState <= STATE_T_SEND_KEY;
                else
                    NextState <= STATE_IDLE;
                end if;
            when STATE_START_DACCEPT =>
                NextState <= STATE_DACCEPT_ACK;
            when STATE_DACCEPT_ACK =>
                NextState <= STATE_AFTER_WRITE;
        end case;
    end process GenerateNextState;

end architecture behavioral;
```

```
when STATE_AFTER_WRITE =>
    NextState <= STATE_WAIT_DAV_LOW;
when STATE_WAIT_DAV_LOW =>
    if DAV = '0' then
        NextState <= STATE_IDLE;
    else
        NextState <= STATE_WAIT_DAV_LOW;
    end if;
when STATE_T_SEND_KEY =>
    NextState <= STATE_T_INCG;
when STATE_T_INCG =>
    NextState <= STATE_IDLE;
end case;
end process GenerateNextState;

incP <= '1' when CurrentState = STATE_AFTER_WRITE else '0';
DAC <= '1' when CurrentState = STATE_AFTER_WRITE or CurrentState = STATE_WAIT_DAV_LOW else '0';
Wr <= '1' when CurrentState = STATE_DACCEPT_ACK else '0';
selPnG <= '1' when CurrentState = STATE_START_DACCEPT or CurrentState = STATE_DACCEPT_ACK else '0';

Wreg <= '1' when CurrentState = STATE_T_SEND_KEY else '0';
incG <= '1' when CurrentState = STATE_T_INCG else '0';

end architecture behavioral;
```

B *Kotlin*

B.1 HAL

Algoritmo 1: HAL - Hardware Abstract Layer

```
import isel.leic.UsbPort

object HAL {
    private var bufferWrite = 0

    fun init() {
        UsbPort.write(bufferWrite)
    }

    fun isBit(mask: Int): Boolean {
        return (UsbPort.read() and mask) != 0
    }

    fun readBits(mask: Int): Int {
        return UsbPort.read() and mask
    }

    fun writeBits(mask: Int, value: Int) {
        bufferWrite = (bufferWrite and mask.inv()) or (value and mask)
        UsbPort.write(bufferWrite)
    }

    fun setBits(mask: Int) {
        bufferWrite = bufferWrite or mask
        UsbPort.write(bufferWrite)
    }

    fun clrBits(mask: Int) {
        bufferWrite = bufferWrite and mask.inv()
        UsbPort.write(bufferWrite)
    }
}
```

B.2 KBD

Algoritmo 2: KBD

```
object KBD {

    // -----
    // Key definitions
    // Maps 4-bit hardware key codes to characters.
    //
    // Code layout from Key_Scan (col[1:0] = column, row[1:0] = row):
    //
    //      COL0    COL1    COL2    COL3
    // ROW0  0000    0100    1000    1100
    // ROW1  0001    0101    1001    1101
    // ROW2  0010    0110    1010    1110
    // ROW3  0011    0111    1011    1111
    //
    // Physical keypad layout:
    //      [ 1 ][ 2 ][ 3 ][ A ]
    //      [ 4 ][ 5 ][ 6 ][ B ]
    //      [ 7 ][ 8 ][ 9 ][ C ]
    //      [ * ][ 0 ][ # ][ D ]
    // -----
    enum class Key(val char: Char) {
        KEY_1('1'),
        KEY_2('2'),
        KEY_3('3'),
        KEY_4('4'),
        KEY_5('5'),
        KEY_6('6'),
        KEY_7('7'),
        KEY_8('8'),
        KEY_9('9'),
        KEY_0('0'),
        KEY_A('A'),
        KEY_B('B'),
        KEY_C('C'),
        KEY_D('D'),
        KEY_HASH('#'),
        KEY_STAR('*'),
        KEY_NONE('\u0000')
    }

    private const val TXD_MASK = 0b10000000 // input port bit 1: TXD from hardware
    private const val TXCLK = 0b10000000 // output port bit 3: serial clock to hardware

    private const val MAX_CLOCKS = 7

    fun init() {
        HAL.clrBits(TXCLK)
        while (readKey() != Key.KEY_NONE) {
            println("ignored key")
        }
    }

    fun getKey(): Char = readKey().char

    fun waitKey(timeout: Long): Char {
        val stopTime = System.currentTimeMillis() + timeout
        while (System.currentTimeMillis() < stopTime) {
            val key = getKey()
            if (key != Key.KEY_NONE.char) return key
            Thread.sleep(1)
        }
        return Key.KEY_NONE.char
    }

    fun readKey(): Key {
        var clockCounter = 0
        var rawCode = -1

        // Check INIT bit: TXD must be LOW to indicate a frame is starting
    }
}
```

```

    if (!HAL.isBit(TXD_MASK)) {
        clockCycle(clockCounter++) // Slot 0: INIT - advance past the low

        // Check START bit: TXD must be HIGH
        if (HAL.isBit(TXD_MASK)) {
            clockCycle(clockCounter++) // Slot 1: START bit confirmed
            rawCode = 0

            // Read 4 data bits, LSB first (K[0]..K[3])
            for (i in 0..3) {
                if (HAL.isBit(TXD_MASK)) {
                    rawCode = rawCode or (1 shl i)
                }
                clockCycle(clockCounter++) // Slots 2..5: data bits
            }

            // Check STOP bit: TXD should be LOW
            if (!HAL.isBit(TXD_MASK)) {
                clockCycle(clockCounter++) // Slot 6: STOP bit
            }
        }

        // Drain remaining clocks so KeyTransmitter finishes cleanly
        while (clockCounter < MAX_CLOCKS) {
            clockCycle(clockCounter++)
        }
    }

    return if (rawCode != -1 && rawCode in 0..15)
        convertToKey(rawCode)
    else
        Key.KEY_NONE
}

private fun clockCycle(count: Int) {
    HAL.setBits(TXCLK)
    HAL.clrBits(TXCLK)
}

private fun convertToKey(code: Int): Key = when (code) {
    0b0000 -> Key.KEY_1
    0b0100 -> Key.KEY_2
    0b1000 -> Key.KEY_3
    0b1100 -> Key.KEY_A
    0b0001 -> Key.KEY_4
    0b0101 -> Key.KEY_5
    0b1001 -> Key.KEY_6
    0b1101 -> Key.KEY_B
    0b0010 -> Key.KEY_7
    0b0110 -> Key.KEY_8
    0b1010 -> Key.KEY_9
    0b1110 -> Key.KEY_C
    0b0011 -> Key.KEY_STAR
    0b0111 -> Key.KEY_0
    0b1011 -> Key.KEY_HASH
    0b1111 -> Key.KEY_D
    else -> Key.KEY_NONE
}
}

```

B.3 SerialEmitter

Algoritmo 3: SerialEmitter

```

object SerialEmitter {
    enum class Peripheral {LCD, TICKET}

    private const val SDX_MASK = 0b00000001
    private const val SCLK_MASK = 0b00000010
    private const val SSLCD_MASK = 0b00000100
    private const val SSTD_MASK = 0b00001000

    fun init() {

```

```

    HAL.init()
    HAL.clrBits(SDX_MASK or SCLK_MASK or SSLCD_MASK or SSTD_MASK)
}

fun send(addr: Peripheral, data: Int, size: Int) {
    var _data = data

    HAL.clrBits(SCLK_MASK)

    if (addr == Peripheral.LCD) {
        HAL.clrBits(SSLCD_MASK)
    } else if (addr == Peripheral.TICKET) {
        HAL.clrBits(SSTD_MASK)
    }

    for (i in 0 until size) {
        HAL.writeBits(SDX_MASK, _data)
        _data = _data.shr(1)
        Thread.sleep(0, 40) // deixa SDX estabilizar
        HAL.setBits(SCLK_MASK)
        Thread.sleep(0, 40) // clock high time
        HAL.clrBits(SCLK_MASK)
    }

    if (addr == Peripheral.LCD) {
        HAL.setBits(SSLCD_MASK) // SS=1
        Thread.sleep(1) // tempo suficiente
        HAL.setBits(SCLK_MASK) // pulso de clock extra para HoldRegister capturar
        HAL.clrBits(SCLK_MASK)
        Thread.sleep(1)
        HAL.clrBits(SSLCD_MASK)
    } else if (addr == Peripheral.TICKET) {
        HAL.setBits(SSTD_MASK) // SS=1
        Thread.sleep(1) // tempo suficiente
        HAL.setBits(SCLK_MASK) // pulso de clock extra para HoldRegister capturar
        HAL.clrBits(SCLK_MASK)
        Thread.sleep(1)
        HAL.clrBits(SSTD_MASK)
    }
}
}
}

```

C Atribuições

C.1 Atribuições do USBPort

Import	pino
CoinId(0)	I0
CoinId(1)	I1
CoinId(2)	I2
Coin	I3
Fn	I4
-	I5
M	I6
TXd	I7
Outport	pino
SDX	O0
SCLK	O1
SSLCD	O2
SSTD	O3
accept	O4
eject	O5
collect	O6
TXclk	O7

Table 1: Atribuição de variáveis para o USBPort